

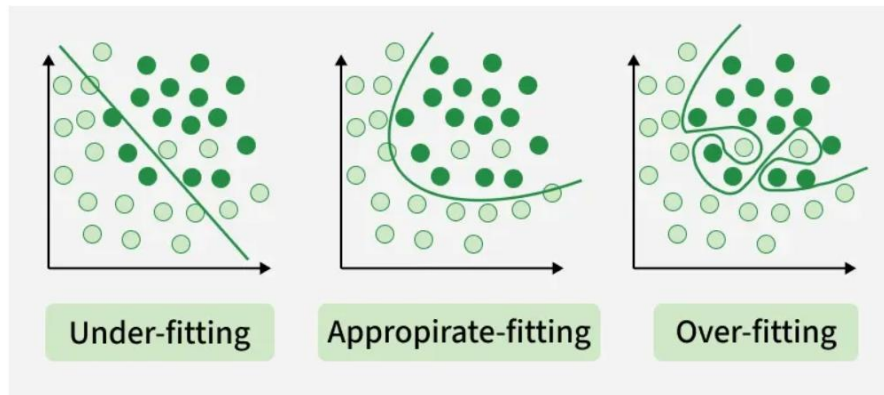
## Contents

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Regularization in Machine Learning .....</b> | <b>2</b>  |
| <b>What is Lasso Regression .....</b>           | <b>7</b>  |
| <b>Implementation .....</b>                     | <b>9</b>  |
| <b>Ridge Regression.....</b>                    | <b>16</b> |

## Regularization in Machine Learning

Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.

- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.



Regularization in Machine Learning

### Types of Regularization

There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.

#### 1. Lasso Regression

A regression model which uses the L1 Regularization technique is called [LASSO \(Least Absolute Shrinkage and Selection Operator\)](#) regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

Where

- $m$ : Number of Features
- $n$ : Number of Examples
- $y_i$ : Actual Target Value
- $\hat{y}_i$ : Predicted Target Value

**Note:** These formulas apply to linear models. In neural networks, the number of weights is much larger than the number of features, but the same regularization principles (L1, L2) still apply on all weights.

Lets see how to implement this using python:

- **X, y = make\_regression(n\_samples=100, n\_features=5, noise=0.1, random\_state=42):** Generates a regression dataset with 100 samples, 5 features and some noise.
- **X\_train, X\_test, y\_train, y\_test = train\_test\_split(X, y, test\_size=0.2, random\_state=42):** Splits the data into 80% training and 20% testing sets.
- **lasso = Lasso(alpha=0.1):** Creates a Lasso regression model with regularization strength alpha set to 0.1.

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")

print("Coefficients:", lasso.coef_)
```

Output:

```
Mean Squared Error: 0.06362439921332456
Coefficients: [60.50305581 98.52475354 64.3929265 56.96061238 35.52928502]
```

*Lasso Regression*

The output shows the model's prediction error and the importance of features with some coefficients reduced to zero due to L1 regularization.

## 2. Ridge Regression

A regression model that uses the L2 regularization technique is called [Ridge regression](#). It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Where,

- $n$ : Number of examples or data points
- $m$ : Number of features i.e predictor variables
- $y_i$ : Actual target value for the  $i$ th example
- $\hat{y}_i$ : Predicted target value for the  $i$ th example
- $w_i$ : Coefficients of the features
- $\lambda$ : Regularization parameter that controls the strength of regularization

Lets see how to implement this using python:

- **ridge = Ridge(alpha=1.0)**: Creates a Ridge regression model with regularization strength alpha set to 1.0.

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
print("Coefficients:", ridge.coef_)
```

Output:

```
Mean Squared Error: 4.114050771972589
Coefficients: [59.87954432 97.15091098 63.24364738 56.31999433 35.34591136]
```

*Ridge Regression*

The output shows the MSE showing model performance. Lower MSE means better accuracy. The coefficients reflect the regularized feature weights.

### 3. Elastic Net Regression

[Elastic Net Regression](#) is a combination of both L1 as well as L2 regularization. That shows that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left( (1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$

Where

- $n$ : Number of examples (data points)
- $m$ : Number of features (predictor variables)
- $y_i$ : Actual target value for the  $i^{th}$  example
- $\hat{y}_i$ : Predicted target value for the  $i^{th}$  example
- $w_i$ : Coefficients of the features
- $\lambda$ : Regularization parameter that controls the strength of regularization
- $\alpha$ : Mixing parameter where  $0 \leq \alpha \leq 1$  and  $\alpha=1$  corresponds to Lasso ( $L_1$ ) regularization,  $\alpha=0$  corresponds to Ridge ( $L_2$ ) regularization and Values between 0 and 1 provide a balance of both L1 and L2 regularization

Lets see how to implement this using python:

- **model = ElasticNet(alpha=1.0, l1\_ratio=0.5)** : Creates an Elastic Net model with regularization strength alpha=1.0 and L1/L2 mixing ratio 0.5.

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = ElasticNet(alpha=1.0, l1_ratio=0.5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Coefficients:", model.coef_)
```

Output:

```
Mean Squared Error: 7785.886176938014
Coefficients: [16.84528938 31.77080959  4.05901996 40.18486737 57.25856154 45.81463318
 58.97979422 -0.          3.82816854 41.1096051 ]
```

Elastic Net Regression

The output shows MSE which measures how far off predictions are from actual values (lower is better) and coefficients show feature importance.

## Benefits of Regularization

Now, let's see various benefits of regularization which are as follows:

- **Prevents Overfitting:** Regularization helps models focus on underlying patterns instead of memorizing noise in the training data.
- **Enhances Performance:** Prevents excessive weighting of outliers or irrelevant features helps in improving overall model accuracy.
- **Stabilizes Models:** Reduces sensitivity to minor data changes which ensures consistency across different data subsets.
- **Prevents Complexity:** Keeps model from becoming too complex which is important for limited or noisy data.
- **Handles Multicollinearity:** Reduces the magnitudes of correlated coefficients helps in improving model stability.
- **Promotes Consistency:** Ensures reliable performance across different datasets which reduces the risk of large performance shifts.

## What is Lasso Regression

Lasso Regression (Least Absolute Shrinkage and Selection Operator) is a linear regression technique with L1 regularization that improves model generalization by adding a penalty. This penalty not only controls overfitting but also performs automatic feature selection by shrinking some coefficients exactly to zero, making Lasso useful in high-dimensional datasets where interpretability and sparsity are important.

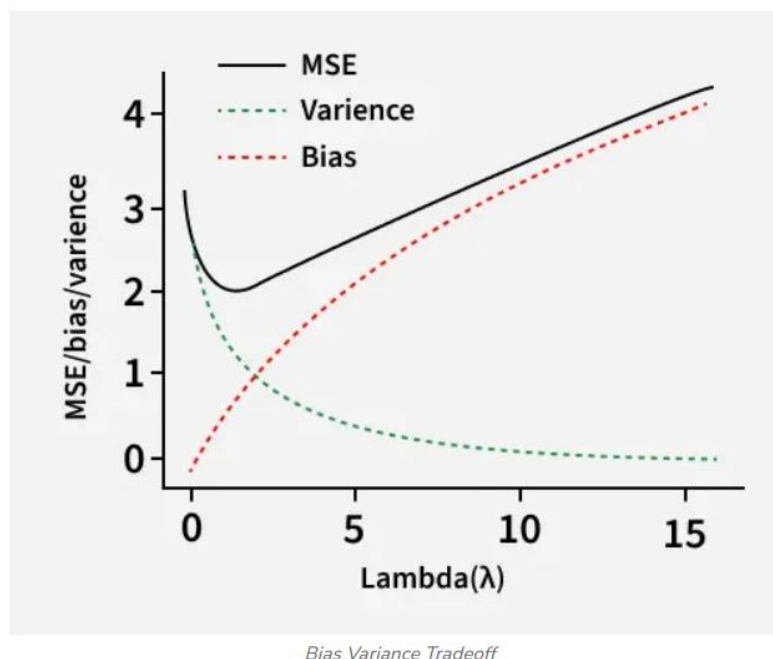
- Uses L1 regularization to reduce overfitting
- Encourages sparse models (zero coefficients)
- Automatically performs feature selection
- Useful for high-dimensional datasets

**Note:** L1 regularization adds the sum of absolute values of model weights to the loss function, encouraging sparsity by pushing some weights exactly to zero.

### **Bias–Variance Tradeoff**

The Bias–Variance Tradeoff describes the balance between a model's simplicity and its ability to generalize to unseen data. In machine learning, reducing bias often increases variance and vice versa, so finding the right balance is important for building accurate models.

Lasso Regression directly influences the bias–variance tradeoff through its regularization strength ( $\lambda$  or  $\alpha$ ).



### **Low $\lambda$ (weak regularization)**

- Model closely fits training data

- Means low bias and high variance
- Risk of overfitting

### High $\lambda$ (strong regularization)

- Coefficients heavily shrunk or removed
- Means higher bias and lower variance
- Better generalization

### How Lasso balances this tradeoff

- Penalization reduces variance by simplifying the model
- Feature elimination may increase bias slightly
- Net effect is often lower test error

### Working of Lasso Regression

Let's see the working:

#### 1. Base Linear Model

Lasso begins with a standard linear regression setup where the target variable is predicted as a weighted sum of input features. Without regularization, this model can overfit when the number of predictors is large.

#### 2. L1-Regularized Objective Function

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |w_j|$$

- First term measures prediction error
- Second term penalizes absolute coefficient values
- $\lambda$  determines how strongly coefficients are constrained

#### 3. Coefficient Shrinkage

- As  $\lambda$  increases, coefficients are gradually reduced
- Less important features receive stronger shrinkage
- Some coefficients reach exactly zero

#### 4. Embedded Feature Selection

- Zero-valued coefficients mean features are discarded

- No separate feature selection step is needed
- Results in compact and interpretable models

## 5. Optimization Strategy

- Typically solved using coordinate descent
- Updates one coefficient at a time
- Efficient even for datasets with many features

### Implementation

Let's implement it in python:

#### Step 1: Import Libraries

We need to import the necessary libraries such as [NumPy](#), [Matplotlib](#) and [sklearn](#).

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import make_regression

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Lasso

from sklearn.metrics import mean_squared_error, r2_score
```

#### Step 2: Sample Dataset

Here we:

- Creates 500 data samples
- Uses 10 input features, out of which only 5 are truly informative
- Adds noise to simulate real-world data
- `random_state` ensures reproducibility
- Helps demonstrate how Lasso removes irrelevant features

```
X, y = make_regression(
    n_samples=500,
    n_features=10,
    n_informative=5,
```

```
noise=15,  
random_state=42  
)
```

### Step 3: Split Data into Training and Testing Sets

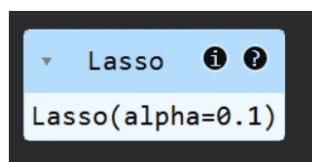
- Splits data into 80% training and 20% testing
- Training data is used to learn coefficients
- Testing data is used to evaluate generalization
- Prevents biased performance estimation

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

### Step 4: Train the Lasso Regression Model

- alpha controls the strength of L1 regularization
- Higher alpha means stronger shrinkage of coefficients
- Model learns optimal coefficients using training data
- Some coefficients may become exactly zero

```
lasso = Lasso(alpha=0.1)  
lasso.fit(X_train, y_train)
```



Model Training

### Step 5: Make Predictions and Evaluate Performance

- Uses trained model to predict outputs for test data
- Mean Squared Error measures average prediction error

- $R^2$  score measures explained variance
- Helps assess how well the model generalizes

```
y_pred = lasso.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
mse, r2
```

```
(29.280987296173713, 0.9933960650641204)
```

### Step 6: Regression Line

- Each point represents one test sample
- X-axis shows actual target values
- Y-axis shows predicted values
- Points closer to a diagonal indicate better predictions
- Spread shows the magnitude of prediction error
- Line shows regression line

```
y_pred = lasso.predict(X_test)
```

```
plt.figure(figsize=(6, 6))
```

```
plt.scatter(y_test, y_pred)
```

```
min_val = min(y_test.min(), y_pred.min())
```

```
max_val = max(y_test.max(), y_pred.max())
```

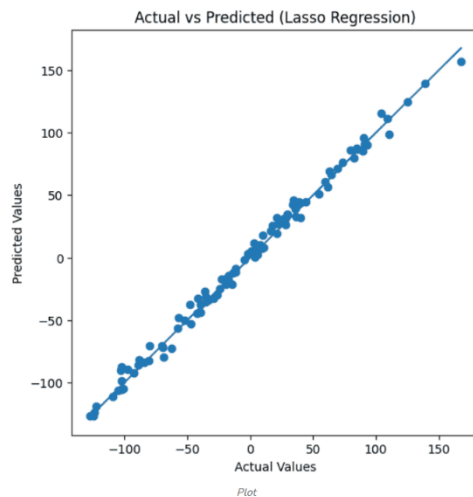
```
plt.plot([min_val, max_val], [min_val, max_val])
```

```
plt.xlabel("Actual Values")
```

```
plt.ylabel("Predicted Values")
```

```
plt.title("Actual vs Predicted (Lasso Regression)")
```

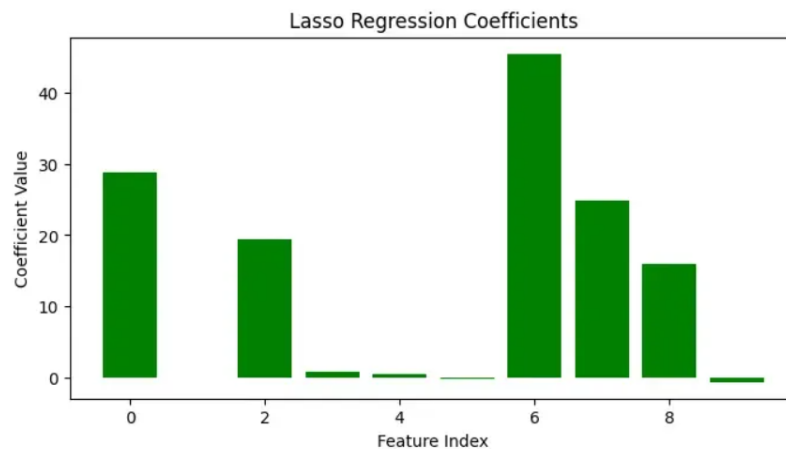
```
plt.show()
```



### Step 7: Plot Lasso Coefficients (Feature Selection)

- Each bar represents one feature's coefficient
- Zero-height bars indicate discarded features
- Non-zero bars indicate selected important features
- Clearly visualizes Lasso's feature selection behavior

```
plt.figure(figsize=(8, 4))
plt.bar(range(len(lasso.coef_)), lasso.coef_, color="green")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.title("Lasso Regression Coefficients")
plt.show()
```



It shows which feature is more important for feature selection.

## Step 8: Plot Effect of Regularization Strength (Lasso Path)

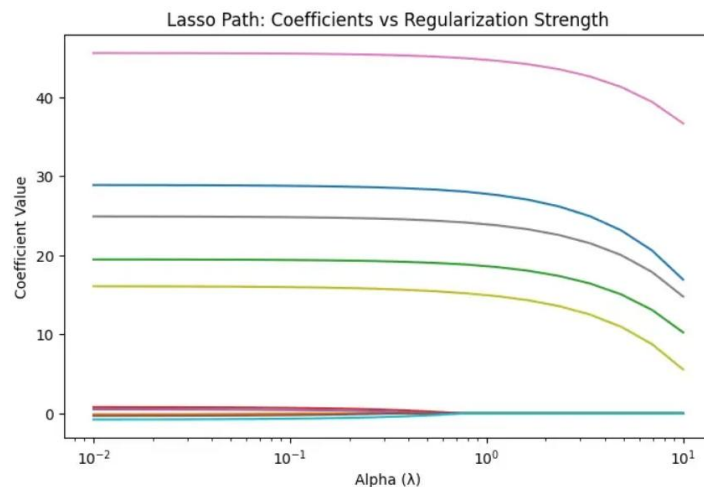
- Generates a range of alpha ( $\lambda$ ) values on a logarithmic scale
- Trains a new Lasso model for each alpha value
- Stores coefficient values for every feature
- Plots how coefficients change as regularization increases
- Shows that coefficients gradually shrink and drop to zero
- Helps identify stable and important features

```
alphas = np.logspace(-2, 1, 20)
coefficients = []

for a in alphas:
    model = Lasso(alpha=a)
    model.fit(X_train, y_train)
    coefficients.append(model.coef_)

coefficients = np.array(coefficients)

plt.figure(figsize=(8, 5))
for i in range(coefficients.shape[1]):
    plt.plot(alphas, coefficients[:, i])
plt.xscale("log")
plt.xlabel("Alpha ( $\lambda$ )")
plt.ylabel("Coefficient Value")
plt.title("Lasso Path: Coefficients vs Regularization Strength")
plt.show()
```



## Result

- Each curve represents one feature's coefficient in the Lasso model
- X-axis shows regularization strength ( $\alpha$ ) on a logarithmic scale
- As  $\alpha$  increases, coefficients shrink toward zero
- Less important features drop to exactly zero
- Important features remain non-zero for longer
- It shows Lasso's feature selection and overfitting control

## Ridge Regression vs Lasso Regression

Let's see the comparison between [ridge regression](#) and lasso regression:

| Aspect                | Ridge Regression                       | Lasso Regression              |
|-----------------------|----------------------------------------|-------------------------------|
| Regularization        | L2 penalty                             | L1 penalty                    |
| Feature selection     | It does not supports feature selection | It supports feature selection |
| Coefficient shrinkage | Smooth shrinkage                       | Aggressive shrinkage          |
| Zero coefficients     | Never                                  | Possible                      |
| Model sparsity        | Dense                                  | Sparse                        |
| Multicollinearity     | Handles very well                      | Moderate handling             |
| Interpretability      | Lower                                  | Higher                        |

## Applications

- **Feature Selection in ML:** Removes irrelevant predictors automatically
- **Genomics:** Selects influential genes from thousands of variables
- **Finance:** Identifies key risk factors and predictors
- **Text Mining:** Chooses important words in high-dimensional text data

- **Econometrics:** Builds compact explanatory models

### Advantages

- **Automatic Feature Selection:** No extra preprocessing needed
- **Sparse Models:** Easier interpretation
- **Overfitting Reduction:** Improves generalization
- **High-Dimensional Suitability:** Works well when features are many
- **Computational Efficiency:** Fast optimization methods

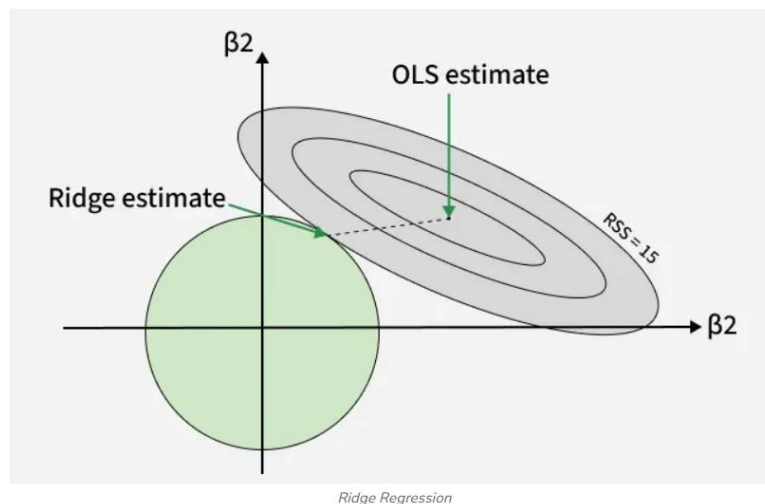
### Limitations

- **Correlated Feature Issue:** Selects one feature from a group arbitrarily
- **Bias Introduction:** Strong regularization may underfit
- **Hyperparameter Sensitivity:**  $\lambda$  must be tuned carefully
- **Limited Group Selection:** Not ideal for grouped predictors
- **Linear Assumption:** Cannot model complex non-linear patterns

## Ridge Regression

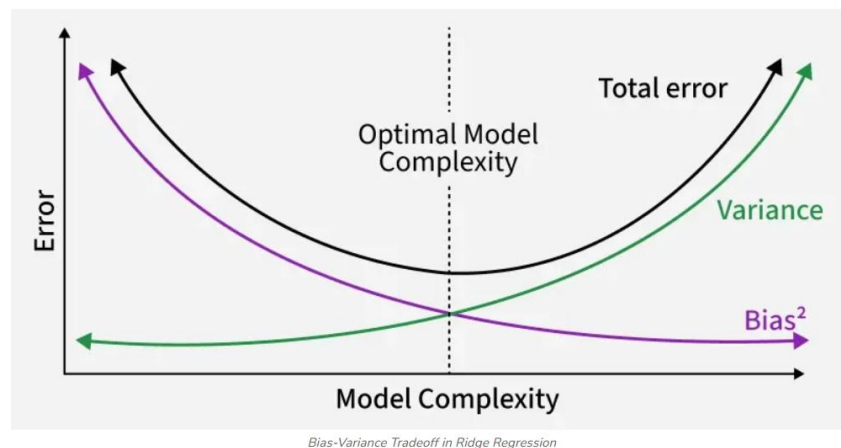
Ridge Regression is a version of linear regression that adds an L2 penalty to control large coefficient values. While Linear Regression only minimizes prediction error, it can become unstable when features are highly correlated. Ridge solves this by shrinking coefficients making the model more stable and reducing overfitting. It helps in:

- **L2 Regularization:** Adds an L2 penalty to model weights
- **Bias-Variance Tradeoff:** Controls how large coefficients can grow
- **Multicollinearity:** Improves stability when features overlap
- **Generalization:** Helps the model generalize better on new data



### Bias-Variance Trade-off in Ridge Regression

One of the central ideas behind ridge regression is the bias-variance trade-off:



- **Variance:** In standard linear regression, especially when features are correlated or many, coefficient estimates can vary a lot depending on the specific training data, meaning predictions on new data can be very unstable.
- **Bias:** Ridge regression deliberately introduces some bias by shrinking coefficient magnitudes. This means the fit to the training data might be slightly worse.
- **Trade-off & Why It Helps:** Ridge shrinks large coefficients hence reducing variance. Even with a small increase in bias, the overall MSE drops, giving better performance than plain linear regression on new data.

Thus, ridge regression accepts a small increase in bias to gain a larger reduction in variance and this tradeoff is often useful when generalization is important.

## Selection of the Ridge Parameter

Choosing the right ridge parameter  $k$  is essential because it directly affects the model's bias-variance balance and overall predictive accuracy. Several systematic approaches exist for determining the optimal value of  $k$ , each offering unique strengths and considerations. The major methods are:

### 1. Cross-Validation

Cross-validation selects the ridge parameter by repeatedly training and testing the model on different subsets of data and identifying the value of  $k$  that minimizes validation error.

- **K-Fold Cross-Validation:** The dataset is divided into  $K$  folds. The model trains on  $K-1$  folds and validates on the remaining fold. This process repeats for all folds and the average error determines the best  $k$ .
- **Leave-One-Out Cross-Validation (LOOCV):** A special form of cross-validation where each observation acts once as the validation point. Though computationally expensive, it provides an almost unbiased estimate of prediction error.

### 2. Generalized Cross-Validation (GCV)

It is an efficient alternative to LOOCV that avoids explicitly splitting the data. It estimates the optimal  $k$  by minimizing a function that approximates the LOOCV error.

- Requires fewer computations.
- Often produces results similar to traditional cross-validation.

### 3. Information Criteria

Model selection metrics like AIC and BIC can also guide the choice of  $k$ .

- They balance model fit with complexity.
- Higher penalties discourage overly complex or over-regularized models.

### 4. Empirical Bayes Methods

These methods treat  $k$  as a Bayesian hyperparameter and use observed data to estimate its value.

- **Empirical Bayes Estimation:** A prior distribution is assigned to  $k$  and the data are used to update it into a posterior distribution. The posterior mean or mode is then selected as the optimal  $k$ .

## 5. Stability Selection

Stability selection enhances robustness by repeatedly fitting the model on subsampled datasets.

- The ridge parameter that appears most consistently across subsamples is chosen.
- Helps avoid unstable or overly sensitive parameter choices.

## Implementation

Let's implement it and here we will import [numpy](#) and [scikit learn](#).

- **StandardScaler:** Ensures all features are on the same scale so the ridge penalty affects them fairly.
- **alpha ( $\lambda$ ) in Ridge Regression:** Controls the strength of regularization hence larger values shrink coefficients more aggressively.
- **Train-test split:** Divides data into training and testing sets to evaluate generalization performance.
- **Ridge(alpha=1.0):** Initializes a Ridge Regression model with a chosen regularization parameter.
- **ridge.fit():** Fits the model to training data and estimates the coefficients under L2 penalty.
- **ridge.predict():** Uses the trained model to generate predictions on unseen test data.
- **mean\_squared\_error():** Measures the average squared difference between predicted and actual values; lower is better.
- **GridSearchCV:** Systematically tests multiple alpha values to find the one that minimizes error using cross-validation.
- **best\_estimator\_:** The model with the optimal alpha found through GridSearchCV.
- **coef\_:** The final learned coefficients after applying L2 shrinkage.

```

import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import mean_squared_error

np.random.seed(0)
X = np.random.randn(200, 6)
true_coef = np.array([3.2, -1.5, 0.7, 0, 2.8, -0.5])
y = X.dot(true_coef) + np.random.randn(200) * 0.6

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=42
)
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
pred_basic = ridge.predict(X_test)

print("MSE (alpha = 1.0):", mean_squared_error(y_test, pred_basic))
print("Coefficients (alpha = 1.0):", ridge.coef_)
param_grid = {"alpha": [0.001, 0.01, 0.1, 1, 10, 100, 500]}
grid = GridSearchCV(Ridge(), param_grid, cv=5,
                    scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)
best_ridge = grid.best_estimator_
pred_best = best_ridge.predict(X_test)

print("Best alpha selected:", grid.best_params_["alpha"])
print("MSE (best alpha):", mean_squared_error(y_test, pred_best))
print("Coefficients (best alpha):", best_ridge.coef_)

```

Output:

```

MSE (alpha = 1.0): 0.45980230236924924
Coefficients (alpha = 1.0): [ 2.96587537 -1.64397225  0.66532484  0.06289239  2.83625348 -0.46900939]
Best alpha selected: 0.001
MSE (best alpha): 0.45099557660691597
Coefficients (best alpha): [ 2.9849965 -1.65395975  0.67045849  0.06557375  2.85454007 -0.47306529]

```

## Applications

- **Multicollinearity Handling:** Stabilizes estimates when predictors are highly correlated.
- **High-Dimensional Data:** Performs well when the number of features is large or exceeds observations.

- **Noise-Resistant Modeling:** Reduces variance and improves prediction on noisy datasets.
- **Domain Usage:** Common in finance, econometrics, genomics and marketing analytics.
- **ML Workflow Integration:** Serves as a strong baseline regularized regression model.

### Advantages

- **Overfitting Control:** Shrinks coefficients to prevent the model from memorizing noise.
- **Correlation Support:** Handles correlated predictors more effectively than linear regression.
- **Better Generalization:** Produces stable predictions on new, unseen data.
- **Feature Retention:** Keeps all features in the model instead of dropping any unlike Lasso.

### Limitations

- **No Feature Selection:** Coefficients are shrunk but never reduced to exact zero.
- **Hyperparameter Sensitivity:** Requires careful  $\lambda$  (alpha) tuning for best performance.
- **Irrelevant Feature Impact:** May still be affected when many inputs add no useful information.
- **Reduced Interpretability:** Heavy shrinkage can obscure the true effect of predictors.
- **Poor Fit for Sparse Models:** Not ideal when only a few predictors truly matter.

### References:

1. <https://www.geeksforgeeks.org/machine-learning/regularization-in-machine-learning/>
2. <https://www.geeksforgeeks.org/machine-learning/what-is-lasso-regression/>
3. <https://www.geeksforgeeks.org/machine-learning/what-is-ridge-regression/>
4. <https://www.geeksforgeeks.org/machine-learning/implementation-of-elastic-net-regression-from-scratch/>